BuildAdvisor – Final Project Presentation

Role-wise Speaking Scripts (Hinglish)
Huzaifa Bangdiwala – Frontend | Krishna Daruwala – Backend | Haasil Dumsiya – Database

Presentation ka order (total ~20–25 min)

#	Part	Kaun bolega	Time
1	Introduction, problem statement, architecture	Huzaifa	3 min
2	Frontend (Streamlit UI) + live demo	Huzaifa	5–6 min
3	Backend (Flask API, logic, AI, pricing) + control flow	Krishna	6–7 min
4	Database (SQLite, E-R, DFD data stores)	Haasil	5–6 min
5	Testing summary, limitations, future scope, conclusion	Krishna (testing: sab milke)	3 min
6	Viva / Q&A	Jis layer ka sawaal, wahi jawab de	–

Note: Scripts ko ratna nahi hai – flow samajh ke apne words mein bolna. **Bold** words key points hain jo zaroor bolne hain.

Part 1 – Introduction (Huzaifa)

~3 minute · Slide: Title, Problem Statement, Objectives, Architecture

> "Good morning respected faculty members aur mere dosto. Main **Huzaifa Bangdiwala**, aur mere saath hain **Krishna Daruwala** aur **Haasil Dumsiya**. Aaj hum apna final year project present karne ja rahe hain jiska naam hai **BuildAdvisor – an AI-Powered PC Build Recommender for Gamers**. Yeh project humne Ms. Falguni Dhabi ma'am ke guidance mein banaya hai."
>
> "Pehle problem samajhte hain. Aaj kal bahut log gaming PC banana chahte hain, lekin ek PC mein 7 alag parts hote hain – CPU, GPU, Motherboard, RAM, Storage, Power Supply aur Case. Ek normal user ko pata nahi hota ki **kaunsa CPU kaunse motherboard mein fit hoga**, kitne watt ki PSU chahiye, ya apne budget mein best GPU kaunsa hai. Galat part le liya to PC boot hi nahi hoga, ya paisa waste hoga. YouTube aur forums pe information bikhri hui hai aur technical language mein hai."
>
> "Isliye humne BuildAdvisor banaya. User sirf **teen cheezein** deta hai – apna **budget**, **target game**, aur **resolution** jaise 1080p, 1440p ya 4K. Aur system

> usko deta hai:
>
> - ek **complete 7-part build** jo budget ke andar ho,
> - jo **fully compatible** ho – socket, RAM type, power aur case size sab check hote hain,
> - har part ka **price** aur retailer,
> - us game mein **estimated FPS**,
> - aur **Google Gemini AI** se simple language mein explanation ki yeh build kyun sahi hai."
>
> "Iske alawa teen aur features hain – **Upgrade Advisor** jo existing PC ka bottleneck batata hai, **Save & Share** aur **Community** jahan saved builds ko rating mil sakti hai, aur ek **Admin Dashboard** jahan admin components aur games manage karta hai."
>
> "Ab architecture. Humara project ek **three-tier client-server architecture** follow karta hai:
>
> - **Presentation layer** – Streamlit frontend, jo maine banaya hai,
> - **Application layer** – Flask REST API aur business logic, jo Krishna ne banaya hai,
> - **Data layer** – SQLite database, jo Haasil ne design kiya hai.
>
> Sab kuch **Python** mein hai. Frontend aur backend HTTP aur JSON ke through baat karte hain, isliye har layer independent hai – ek ko badlo to doosri pe asar nahi padta. Ab main apna part, yaani frontend, explain karta hoon."

Part 2 – Frontend: Streamlit UI (Huzaifa)

~5–6 minute

> Screen par: browser mein app (localhost:8501) · VS Code mein `app.py`, `api.py`, `ui.py`

2.1 Streamlit kyun?

> "Frontend ke liye humne **Streamlit** use kiya hai. Streamlit ek Python framework hai jisse hum **bina HTML, CSS ya JavaScript likhe** interactive web app bana sakte hain. Slider, dropdown, button, table – sab Python functions hain jaise `st.slider`, `st.selectbox`, `st.button`. Streamlit ki khaas baat yeh hai ki jab bhi user koi widget change karta hai, poori script upar se neeche dobara run hoti hai. Isliye jo data yaad rakhna hota hai – jaise last generated build ya admin login token – woh hum **`st.session_state`** mein store karte hain."

2.2 Thin client concept – api.py

> "Humara frontend ek **thin client** hai. Matlab `app.py` mein **koi business logic nahi hai** – na build generate hoti hai, na database touch hota hai. Frontend ka kaam sirf hai: input lena, backend ko bhejna, aur result sundar tarike se dikhana."
>
> "Backend se baat karne ke liye maine ek alag module banaya hai – **`api.py`**. Ismein ek central function hai `_request()` jo `requests` library se HTTP call karta hai, aur uske upar chhote-chhote functions hain jaise `generate_build()`, `upgrade()`, `save_build()`, `list_builds()`, `rate_build()`, `admin_login()`, `admin_add_component()` wagairah. Har function ek backend endpoint se map hota hai."
>
> "Agar backend band ho, ya error aaye, to `_request()` ek custom exception **`BackendError`** raise karta hai. `app.py` mein ek helper `_handle()` hai jo yeh error pakad ke screen pe saaf red error message dikhata hai aur page ko wahi rok deta hai – **app crash nahi hoti**."

2.3 Sidebar – status chips aur navigation

> "Ab app dekhte hain. Left side **sidebar** hai. Sabse upar **status chips** hain – yeh backend ke `/health` endpoint se aate hain. Green 'Backend online' matlab Flask server chal raha hai; 'Gemini live' matlab AI key configured hai; 'Pricing live' ya 'simulated'. Agar backend band ho to red **'Backend offline'** chip dikhta hai aur message aata hai ki `python backend.py` chalao."

> "Neeche navigation hai – chaar screens: **Build Advisor, Upgrade Advisor, Community, Admin**. Aur sabse neeche do toggles hain – **'Live pricing available'** aur **'Gemini API available'**. Inse hum testing ke time outage simulate karte hain, taaki dikha sakein ki service band hone pe bhi app chalti rehti hai."
>
> "Design ke liye maine **`ui.py`** banaya – ismein custom CSS se **dark violet theme** hai, gradient wala `hero()` header, coloured `chip()` badges aur metric cards. Isse UI professional dikhta hai."

2.4 Live Demo – Build Advisor

> "Ab main live demo deta hoon. Build Advisor screen pe maine budget slider se **₹80,000** select kiya, target game **Starfield**, aur resolution **1080p**. Slider ka minimum value aur resolutions ki list bhi hardcoded nahi hai – woh backend ke `/config` endpoint se aati hai, aur games ki list `/games` se."
>
> *[Generate Build dabao]*
>
> "Generate Build dabate hi frontend ek **POST request** bhejta hai `/generate-build` pe, JSON mein budget, game, resolution aur dono toggles ki value. Kuch second mein result aata hai:
>
> - upar **metric cards** – total price, estimated FPS, aur budget mein kitna bacha,
> - neeche **recommended parts** ka table – har part ka naam, tier aur price,
> - **'Per-retailer price breakdown'** expander – har retailer ka price,
> - aur **AI explanation** card, jo Gemini ne simple English mein likha hai.
>
> Agar koi compatibility warning ho to yellow warning box dikhta hai."
>
> "Ab main is build ko naam deta hoon aur **'Save & get share link'** dabata hoon. Green success message aata hai jisme ek unique share code hai. Yeh build ab database mein save ho gaya."

2.5 Baaki screens

> "**Upgrade Advisor** – jinke paas already PC hai. User apne current CPU, GPU, RAM aur extra budget daalta hai, aur system batata hai **'Biggest bottleneck: GPU'** aur us budget mein best upgrade part."
>
> "**Community** – saare saved builds cards mein dikhte hain, total cost, game, aur **average rating** ke saath. Koi bhi user 1 se 5 star rating de sakta hai."
>
> "**Admin** – pehle password se login. Galat password pe 'Incorrect password' error. Login hone ke baad Admin Dashboard – components ki list, naya

> component add karna, naya game add karna, **JSON file se bulk import**, aur spam ratings remove karna."
>
> "Graceful fallback bhi dikhata hoon – 'Gemini API available' toggle off karo, phir bhi build, price aur FPS aate hain, sirf explanation ki jagah message aata hai aur **Retry** button. Aur 'Live pricing' off karo to note aata hai 'showing last-known prices'. Yaani frontend har situation mein user ko clear feedback deta hai."
>
> "Ab frontend se request backend tak kaise jaati hai aur wahan kya hota hai – yeh Krishna explain karega."

Huzaifa – Possible Viva Questions

Q1. Streamlit hi kyun, React kyun nahi?

Poora project Python mein hai; Streamlit se UI Python mein hi ban jaata hai, development fast hota hai, aur data apps ke liye perfect hai. React ke liye alag JavaScript stack seekhna padta.

Q2. session_state kya hai?

Streamlit har interaction pe script re-run karta hai, isliye variables reset ho jaate hain. `st.session_state` ek dictionary hai jo re-run ke beech data yaad rakhti hai – hum isme `last_result` aur `admin_token` rakhte hain.

Q3. Thin client ka fayda?

Logic ek jagah (backend) hai, to future mein mobile app ya doosra frontend same API use kar sakta hai. Testing aur maintenance aasaan.

Q4. Backend band ho to kya hota hai?

`api.py` `BackendError` raise karta hai, sidebar mein red 'Backend offline' chip dikhta hai, app crash nahi hoti (Test Case TC14).

Q5. Frontend aur backend kaise baat karte hain?

HTTP REST calls, JSON format mein, `requests` library se. Backend port 8000 pe, frontend 8501 pe chalta hai.

Part 3 – Backend: Flask API & Logic (Krishna)

~6–7 minute

> Screen par: `backend.py`, `logic.py`, `ai.py`, `pricing.py`, `config.py` · Slide: Control Flow diagram, Level 1 DFD

3.1 Backend ka role

> "Thank you Huzaifa. Main **Krishna Daruwala**, aur maine project ka **backend** banaya hai, jo project ka 'dimaag' hai. Backend **Flask** mein hai – Flask ek lightweight Python web framework hai jisse REST API banate hain. Backend port **8000** pe chalta hai. Frontend jo bhi request bhejta hai, woh yahan aati hai – yahan input validate hota hai, build generate hoti hai, price aur FPS calculate hote hain, AI se explanation aati hai, aur database mein data save hota hai."
>
> "Backend ko maine modules mein toda hai taaki code clean rahe:
>
> - `backend.py` – saare **REST endpoints** (routes),
> - `logic.py` – **core business logic**: build generation, compatibility, FPS, upgrade,
> - `ai.py` – **Google Gemini API** integration,
> - `pricing.py` – **live price scraping**,
> - `config.py` – **business rules** jo `.env` file se aate hain."

3.2 REST API Endpoints

Method	Endpoint	Kaam
GET	/health	Backend, Gemini, pricing, rules aur catalogue ka status
GET	/config	Min budget, tiers, resolutions, retailers
GET	/games, /components	Games aur components ki list
POST	/generate-build	Main feature – build + price + FPS + explanation
POST	/upgrade	Bottleneck aur best upgrade
POST / GET	/builds	Build save karna / saved builds ki list
POST	/builds/<id>/ratings	Rating dena (1–5)
POST	/admin/login	Admin login, token return
POST	/admin/components, /admin/games, /admin/import	Catalogue manage (token required)

DELETE	/admin/builds/<id>/ratings	Ratings remove (token required)

> "Admin wale saare routes ek **decorator** **`require_admin`** se protected hain. Yeh har request ke header mein **`X-Admin-Token`** check karta hai; token galat ya missing ho to **401 Unauthorized** return hota hai. Token sirf sahi password se `/admin/login` karne pe milta hai."

3.3 Main flow – /generate-build step by step

> "Ab sabse important part – jab user Generate Build dabata hai to backend mein kya hota hai. *[Control flow slide dikhao]*
>
> **Step 1 – Validation:** Pehle JSON body se budget nikalta hoon. Agar budget number nahi hai to **400 error** 'Budget must be a number', aur zero ya negative ho to 'Please enter a valid budget'."
>
> "**Step 2 – Data load:** `db.get_catalogue()` se saare components type-wise aate hain aur `db.get_games()` se games. Agar database khaali hai to clear message jaata hai ki pehle Admin se data add karo."
>
> "**Step 3 – Budget tier:** `tier_for_budget()` budget ko tier mein daalta hai – **entry, mid ya high**. Yeh ranges hardcoded nahi hain, `.env` se aati hain, jaise entry 0–55,000, mid 55,000–1,10,000."
>
> "**Step 4 – Budget allocation:** `generate_build()` budget ko 7 parts mein baant-ta hai fixed weights se: **GPU 32%, CPU 20%, Motherboard 13%, RAM 10%, PSU 9%, Storage 8%, Case 8%**. GPU ko sabse zyada isliye kyunki gaming performance mostly GPU pe depend karti hai. Parts is order mein chune jaate hain – GPU, CPU, Motherboard, RAM, Storage, PSU, Case. Har part ke liye ek **reserve** bhi rakha jaata hai – yaani baaki bache parts ki minimum price pehle hi side mein rakh di jaati hai, taaki last mein case ya PSU ke liye paisa khatam na ho. Har slot mein `_pick()` function us budget mein **sabse zyada performance score** wala part chunta hai, preferred tier ko priority deke."
>
> "**Step 5 – Compatibility check:** `check_compatibility()` char rules check karta hai:
>
> - CPU ka **socket** = Motherboard ka socket (jaise AM4, LGA1700),
> - RAM ka **type** = Motherboard ka RAM type (DDR4 / DDR5),
> - PSU wattage ≥ (CPU TDP + GPU TDP) × **1.4** – yaani **40% safety margin**,
> - Motherboard ka **form factor** case mein fit ho (ATX, Micro-ATX)."
>
> "**Step 6 – Repair loop:** Agar koi problem mile to `_repair()` function automatically fix karta hai – **maximum 8 attempts**. Socket mismatch ho to

> CPU ke socket wala sabse sasta motherboard laata hai, RAM mismatch ho to board ke type wala RAM, PSU kam ho to required wattage wala PSU, case fit na ho to compatible case. Iske baad **budget trim** hota hai – agar total budget se upar hai to Storage, RAM, GPU... is order mein part ko agle saste compatible option se replace karta hai, jab tak budget ke andar na aa jaaye. Budget kabhi exceed nahi hona chahiye – yeh humara business rule hai."
>
> "**Step 7 – FPS estimate:** `estimate_fps()` ek benchmark model use karta hai. Agar game **CPU-bound** hai to GPU score ko 55% aur CPU ko 45% weight, warna GPU 80% aur CPU 20%. Phir formula: **FPS = (base × 3.2) ÷ (game weight × resolution multiplier)**. 4K ka multiplier zyada hai, isliye wahan FPS kam aata hai. Minimum 15 FPS, aur result ko 'approximate' mark karte hain."
>
> "**Step 8 – Pricing:** `price_build()` har part ka sabse kam price aur retailer laata hai. Live pricing on ho to `pricing.py` **BeautifulSoup** se retailer sites scrape karta hai, result **cache** karta hai. Agar scraping fail ho ya off ho to **last-known price** ek note ke saath dikhate hain."
>
> "**Step 9 – AI Explanation:** `explain_build()` – agar Gemini API key configured hai to `ai.py` ek **prompt** banata hai jisme budget, game, resolution, FPS aur saare parts hote hain, aur Gemini ke **gemini-2.5-flash** model ko REST call bhejta hai, 20 second timeout ke saath. Agar network error, timeout ya rate limit aaye to **local fallback explanation** template se banti hai. Yeh humara **graceful degradation** hai – AI band ho tab bhi user ko answer milta hai."
>
> "**Step 10 – Response:** Last mein sab kuch ek JSON mein frontend ko jaata hai – build, price rows, total, FPS, explanation, warnings, aur ek **share code**. Share code `share_id()` se banta hai – saare component IDs ko sort karke **SHA-1 hash** ke pehle 10 characters. Isliye same build ka share code hamesha same hota hai, aur duplicate save nahi hota."

3.4 Upgrade, Save, Security, Config

> "**Upgrade Advisor** – `upgrade_advice()` CPU, GPU aur RAM ko normalise karke compare karta hai; jiska score sabse kam, woh **bottleneck**. Phir us type ke parts mein se jo current se better ho aur extra budget mein aaye, unmein best recommend karta hai."
>
> "**Save build** – `/builds` route build name ko **`sanitize()`** karta hai – sirf letters, numbers aur kuch safe characters, max 60 characters – taaki koi galat input store na ho. Rating 1 se 5 ke beech hi accept hoti hai, warna 400 error."
>
> "**Configuration** – `config.py` saare business rules environment variables se padhta hai, jaise `BUILDADVISOR_MIN_BUDGET`, `BUILDADVISOR_TIERS`,

> `BUILDADVISOR_RESOLUTIONS`. Koi value missing ho to app crash nahi hoti, balki clear 'not configured' message deti hai. API key bhi `.env` mein hai, code mein nahi."
>
> "Toh summary: request aati hai **app.py → api.py → backend.py → logic.py → db.py**, saath mein pricing aur AI modules, aur response wapas usi raste jaata hai. Ab yeh saara data kahan aur kaise store hota hai – yeh Haasil batayega."

Krishna – Possible Viva Questions

Q1. Flask kyun, Django kyun nahi?

Humein sirf REST API chahiye tha; Flask lightweight hai, minimal setup, aur Django ka admin/ORM/templates humein zaroori nahi tha.

Q2. REST API kya hai?

Client-server communication ka style jisme resources URLs se identify hote hain aur HTTP methods (GET, POST, DELETE) use hote hain; data JSON mein jaata hai aur server stateless hota hai.

Q3. PSU mein 1.4 kyun?

40% safety headroom – power spikes, future upgrades aur PSU efficiency ke liye; industry mein bhi 30–50% headroom recommend hota hai.

Q4. Repair loop 8 baar hi kyun?

Max 4 rules hain aur ek fix doosri problem create kar sakta hai; 8 attempts kaafi hain aur infinite loop se bachata hai.

Q5. Gemini fail ho to?

`ai.explain()` None return karta hai aur `explain_build()` local template se explanation deta hai (TC07).

Q6. Admin security kaise hai?

Password login se token milta hai, aur har admin route pe `require_admin` decorator `X-Admin-Token` header check karta hai. (Limitation: demo password hai – production mein hashed passwords aur JWT use karenge.)

Q7. FPS exact hai?

Nahi, yeh weighted benchmark model se estimate hai, isliye 'approximate' mark karte hain. Future mein real benchmark data use karenge.

Part 4 – Database: SQLite Design (Haasil)

~5–6 minute

> Screen par: `db.py`, DB Browser for SQLite (`buildadvisor.db`) · Slide: E-R Diagram, Data Dictionary, Level 1 DFD (D1–D7)

4.1 SQLite kyun?

> "Thank you Krishna. Main **Haasil Dumsiya**, aur maine project ka **database** design aur implement kiya hai. Humne **SQLite 3** use kiya hai. SQLite ek **serverless, file-based relational database** hai – poora database ek single file `buildadvisor.db` mein hota hai. Ise alag server install karne ki zaroorat nahi, aur yeh Python ke saath built-in aata hai `sqlite3` module ke roop mein. Humare project ke size ke liye yeh perfect hai, aur phir bhi yeh SQL, primary keys, foreign keys aur transactions sab support karta hai."
>
> "Saara database code ek module **`db.py`** mein hai. Backend kabhi directly SQL nahi likhta – woh sirf `db.py` ke functions call karta hai. Isse data layer alag rehti hai."

4.2 Seven tables

> "Humare database mein **7 tables** hain, jo DFD mein data stores D1 se D7 hain:"

Store	Table	Primary Key	Main Fields / Purpose
D5	User	user_id (AUTOINCREMENT)	name, email, role (default 'Gamer') – users aur admin
D1	Component	component_id (TEXT)	type, name, price, tier, **specs (JSON)** – saare PC parts
D2	Game	game_id (TEXT)	name, weight, cpu_bound – games aur unka load
D3	Build	build_id (AUTOINCREMENT)	share_code (UNIQUE), user_id (FK), name, game, total_price, created_at
D4	BuildComponent	(build_id, component_id)	Build aur Component ka link (junction table)
D6	Benchmark	(build_id, game_id)	estimated_fps – build ka game mein FPS
D7	Rating	rating_id (AUTOINCREMENT)	build_id (FK), user_id, score, comment

> "Ek interesting design decision – **Component table ka `specs` column JSON text** hai. Kyunki har part ke specs alag hote hain – CPU ka socket aur TDP,

> RAM ka type, PSU ka wattage, case ka supported form factor. Agar har spec ka alag column banate to bahut saare NULL columns hote. JSON se ek hi table mein saare types aa jaate hain, aur `_row_to_component()` function read karte waqt JSON ko Python dictionary mein convert kar deta hai. `type` column pe ek **index** bhi banaya hai `idx_component_type`, kyunki hum baar-baar type se filter karte hain."

4.3 E-R Diagram aur Relationships

> *[E-R diagram slide dikhao]* "Yeh humara E-R diagram hai, **Chen notation** mein – rectangle entity, diamond relationship, ellipse attribute, aur underline wala attribute primary key. Relationships:
>
> - **User – creates – Build: 1 to Many**. Ek user kai builds save kar sakta hai.
> - **Build – receives – Rating: 1 to Many**. Ek build ko kai ratings mil sakti hain.
> - **User – gives – Rating: 1 to Many**.
> - **Build – Component: Many to Many**. Ek build mein 7 components hote hain, aur ek component (jaise RTX 4060) kai builds mein ho sakta hai. Many-to-many ko directly store nahi kar sakte, isliye humne **junction table BuildComponent** banayi jiski **composite primary key** (build_id, component_id) hai. Yeh weak entity hai, isliye diagram mein double rectangle hai.
> - **Build – Game via Benchmark** – ek build ke kai games mein FPS ho sakte hain aur ek game ke kai builds mein; Benchmark table bhi composite key (build_id, game_id) use karti hai."

4.4 Integrity – Foreign Keys aur Cascade

> "Data integrity ke liye humne **foreign keys** use ki hain. SQLite mein foreign keys default off hoti hain, isliye `get_conn()` function har connection pe **`PRAGMA foreign_keys = ON`** chalata hai. Saath hi `row_factory = sqlite3.Row` set kiya hai taaki result column name se access ho sake."
>
> "BuildComponent, Benchmark aur Rating mein **`ON DELETE CASCADE`** hai – matlab agar koi build delete ho, to uske components link, benchmark aur ratings automatically delete ho jaate hain. **Orphan records nahi bachte**. `share_code` pe **UNIQUE** constraint hai, aur price, name jaise important fields **NOT NULL** hain."
>
> "Saari queries **parameterized** hain – `?` placeholders use karte hain, string jod ke SQL nahi banate. Isse **SQL injection** se protection milti hai."

4.5 Main functions in db.py

> "**`init_db()`** – app start hote hi chalta hai; `CREATE TABLE IF NOT EXISTS` se saari tables bana deta hai, existing data ko touch nahi karta."
>
> "**`get_catalogue()`** – saare components ko type-wise group karke deta hai, jaise `{'GPU': [...], 'CPU': [...]}` – yahi build generator ka input hai."
>
> "**`save_build()`** – pehle check karta hai ki same `share_code` wala build already hai kya; hai to wahi ID return, duplicate nahi banta. Naya ho to **Build** table mein ek row, **BuildComponent** mein 7 rows `executemany` se, aur agar FPS hai to **Benchmark** mein ek row – sab ek hi connection mein, phir **commit**."
>
> "**`get_builds()`** – Community page ke liye; har build ke components ko **JOIN** query se laata hai – `Component JOIN BuildComponent ON component_id` – aur uski saari rating scores, jinse frontend average rating dikhata hai. Latest build pehle, `ORDER BY created_at DESC`."
>
> "**`add_rating()`**, **`remove_ratings()`**, **`add_component()`**, **`add_game()`** – rating aur admin ke kaam ke liye."

4.6 Data loading

> "Code mein koi hardcoded data nahi hai. Database shuru mein khaali hota hai. Data do tarike se aata hai: ek, **`bootstrap.py`** script jo `catalogue.json` file se saare components aur games import karti hai; doosra, Admin Dashboard ka **JSON bulk import** ya ek-ek component add karna. Abhi catalogue mein **35 components aur 13 games** hain."
>
> *[DB Browser mein Build aur BuildComponent table dikhao]* "Yeh dekhiye – abhi Huzaifa ne jo build save kiya tha, woh Build table mein aa gaya, aur BuildComponent mein uske 7 parts. Isse prove hota hai ki frontend se database tak poora flow kaam kar raha hai. Ab Krishna testing aur conclusion batayega."

Haasil – Possible Viva Questions

Q1. SQLite kyun, MySQL kyun nahi?
Serverless, zero configuration, Python mein built-in, single file – project scale ke liye kaafi. Limitation: bahut zyada concurrent users ke liye nahi; future mein PostgreSQL pe move karenge.

Q2. Primary key aur foreign key mein difference?
Primary key table ki har row ko uniquely identify karti hai; foreign key doosri table ki primary key ko refer karke do tables ko link karti hai aur referential integrity maintain karti hai.

Q3. Junction table kyun?
Many-to-many relationship relational DB mein directly nahi ban sakti; BuildComponent use do one-to-many relationships mein todti hai.

Q4. Composite key kya hai?
Do ya zyada columns milke primary key – jaise (build_id, component_id) – taaki ek build mein same component do baar na aaye.

Q5. Normalization?
Tables 3NF tak normalized hain – components ek hi jagah store, build sirf IDs se refer karta hai, koi repeating group nahi. Specs JSON ek conscious trade-off hai flexibility ke liye.

Q6. ON DELETE CASCADE ka example?
Admin build delete kare to uski Rating, Benchmark aur BuildComponent rows automatically delete ho jaati hain.

Q7. SQL injection se kaise bachaya?
Parameterized queries (`?` placeholders) aur input `sanitize()`.

Part 5 – Testing, Limitations, Future Scope & Conclusion (Krishna)

~3 minute · Slide: Test Cases, Advantages/Limitations, Future Scope, Thank You

> "**Testing** – humne unit testing, integration testing, black-box testing aur fault-tolerance testing ki. Main test cases:
>
> - TC01 – ₹80,000 mid-range build: compatible aur budget ke andar – **Pass**,
> - TC02 – 4K target: stronger GPU choose hua – Pass,
> - TC07 – Gemini band: build aaya, fallback message – Pass,
> - TC08 – Pricing band: last-known prices – Pass,
> - TC09 – Upgrade Advisor ne GPU bottleneck pakda – Pass,
> - TC10 – Save & share – Pass,
> - TC12 – Galat admin password: access denied – Pass,
> - TC14 – Backend offline: red chip, no crash – Pass.
>
> Gemini API ke liye alag script `test_gemini.py` bhi banayi thi."
>
> "**Limitations** – abhi catalogue mein sirf 35 components aur 13 games hain; FPS approximate hai; live scraping by default off hai kyunki retailer websites ka structure badalta rehta hai; full user registration nahi hai, sirf admin login; aur SQLite bahut high traffic ke liye nahi hai."
>
> "**Future scope** – zyada components (coolers, monitors), real benchmark data, price-drop alerts, user login aur build history, laptops aur streaming workloads ka support, PostgreSQL aur cloud deployment, aur Gemini se chat-style assistant."
>
> "**Conclusion** – BuildAdvisor ek complex technical decision ko teen simple inputs mein badal deta hai. Har build compatible aur budget ke andar hai, AI usko simple language mein samjhata hai, aur koi service fail ho tab bhi system chalta rehta hai. Is project se humne REST APIs, relational database design, AI integration aur teamwork practically seekha."
>
> "Hum apni guide **Ms. Falguni Dhabi ma'am** aur saare faculty members ka dil se dhanyavaad karte hain. **Thank you!** Ab hum aapke questions ka jawab dene ke liye ready hain."

Demo Checklist & Backup Plan

Presentation se pehle (setup)

- Laptop charge, charger saath; internet (mobile hotspot backup).
- `.env` file check – business rules aur Gemini key. **Screen pe .env file kabhi mat kholna** (API key dikh jaayegi).
- Terminal 1: `python backend.py` → Terminal 2: `streamlit run app.py` (ya `./run.sh`). Browser mein localhost:8501 pehle se khula rakho.
- Database mein data loaded ho (`python bootstrap.py` agar khaali ho). Ek-do builds pehle se saved rakho taaki Community khaali na dikhe.
- VS Code mein files pehle se tabs mein khol ke rakho: app.py, api.py, backend.py, logic.py, db.py.
- DB Browser for SQLite mein buildadvisor.db khula rakho (Haasil).

Demo order

1. Build Advisor: ₹80,000 · Starfield · 1080p → Generate → Save
2. Same, 4K select karke FPS ka difference dikhao
3. Upgrade Advisor: weak GPU daal ke bottleneck dikhao
4. Community: saved build + rating do
5. Admin: galat password → error; sahi password → dashboard
6. Toggles off karke fallback dikhao
7. DB Browser mein saved build dikhao

Agar kuch fail ho jaaye

- **Internet nahi hai / Gemini fail:** ghabrana nahi – bolo "yahi humara graceful fallback feature hai" aur local explanation dikhao.
- **Backend start nahi ho raha:** red 'Backend offline' chip dikhao (TC14), phir report ke screenshots slides se explain karo.
- **App bilkul na chale:** PPT ke screenshots aur report ke test case figures se demo explain karo.
- Kisi sawaal ka jawab na pata ho: "Yeh humare future scope mein hai" ya teammate ko pass karo – guess mat karo.